

# React Basics: Detailed Notes & Code Reference

## 1. Functional Components

Functional components are JavaScript or TypeScript functions that return JSX/React elements to define the user interface.

- **Why:** Modern React relies on functional components because they are lightweight, easier to read, test, and optimize. Hooks (`useState`, `useEffect`, etc.) allow functions to handle state and lifecycle behavior without needing ES6 classes.
- **When to use:** Use functional components for **all new code**. Class components should only be maintained when working with legacy codebases.
- **How:**

## 2. Props vs. State (`useState`)

Data in React flows through two main constructs: **Props** and **State**.

- **Props (Properties):** Read-only data passed from a parent component down to a child component.
- **State (`useState`):** Internal, mutable memory owned by a single component. Updating state triggers React to re-render the screen.

### Core Comparison

### Code Example

## 3. `useEffect` & Side Effects

`useEffect` allows components to synchronize with external systems or perform side effects (such as fetching data, setting up timers, or listening to network changes).

- **Why:** Render functions must stay pure. Side effects must run **after** React finishes updating the screen.
- **When to use:** Use for API requests, subscriptions, intervals/timeouts, or updating native device features when values change.

### The Dependency Array Rules

The second argument to `useEffect` strictly controls when the effect executes:

1. **No array** (`useEffect(() => {})`): Runs after **every** render.
2. **Empty array** (`useEffect(() => {}, [])`): Runs **once** when the component mounts.
3. **With dependencies** (`useEffect(() => {}, [id])`): Runs on mount and whenever `id` changes.

## Code Example (Data Fetching + Cleanup)

### 4. `useReducer`

`useReducer` is an alternative to `useState` for managing complex state structures or state updates that depend on multiple sub-values.

- **Why:** When state logic gets crowded with multiple `useState` calls, `useReducer` centralizes state transitions into a single function using structured action objects (`{ type, payload }`).
- **When to use:**
  - Complex state objects (e.g., forms with 6+ inputs).
  - State where updates depend on previous values (e.g., shopping carts, step wizards).
  - When state transitions follow specific patterns (`ADD_ITEM`, `REMOVE_ITEM`, `CLEAR`).

## Code Example

### 5. `useCallback` & Performance Optimization

In JavaScript, inline functions are recreated at a new memory location on every render. Passing inline functions to optimized child components breaks memoization (`React.memo`).

- **Why:** `useCallback` caches (memoizes) a function instance in memory across re-renders unless its dependencies change.
- **When to use:**
  - Passing callbacks to optimized child components wrapped in `React.memo`.
  - Formulating `renderItem` handlers for large `FlatList` components to keep scrolling smooth.

## Code Example (`FlatList` Optimization)

### 6. `useContext`

`useContext` provides a way to pass data through the component tree without having to manually pass props down through every level (a problem known as **Prop Drilling**).

- **Why:** Top-level application state (such as light/dark themes, current user authentication, or language preferences) is needed across many unrelated components.

- **When to use:** Use for global/app-wide configuration values that change infrequently.

## Code Example

# Summary Cheat Sheet

Handling responsive layouts in React Native comes down to adapting your UI dynamically to different screen sizes, device orientations (portrait vs. landscape), and pixel densities.

While fixed dimensions (like `width: 200`) work for simple elements, relying on screen dimensions ensures your app scales cleanly across small phones, large flagships, and tablets.

Here is a complete breakdown of how to use `Dimensions` and `useWindowDimensions`, when to use each, and modern best practices.

## 1. `useWindowDimensions` (Recommended)

`useWindowDimensions` is a built-in React Native hook that retrieves the window's width, height, font scale, and pixel ratio.

- **Why:** It **automatically triggers a component re-render** when screen dimensions change (e.g., when a user rotates their device or enters split-screen mode on Android/iPadOS).
- **When to use:** Use this inside **functional components** whenever you need to calculate layouts based on screen size or orientation.

## Code Example

### TypeScript

```
import React from 'react';
import { View, Text, StyleSheet, useWindowDimensions } from 'react-native';

const ResponsiveHeader = () => {
  // Dynamically updates on screen rotation / size changes
  const { width, height } = useWindowDimensions();

  // Determine orientation dynamically
  const isLandscape = width > height;

  return (
    <View style={[styles.container, { padding: width > 600 ? 32 : 16 }]}>
      <Text style={styles.text}>Screen Width: {Math.round(width)}px</Text>
      <Text style={styles.text}>
        Orientation: {isLandscape ? 'Landscape' : 'Portrait'}
      </Text>

      {/* Conditional Layout: 2 Columns on wide screens, 1 Column on narrow screens */}
    </View>
  );
};
```

```

    <View style={{ flexDirection: width > 500 ? 'row' : 'column', gap: 12,
marginTop: 16 }}>
      <View style={styles.card} />
      <View style={styles.card} />
    </View>
  </View>
);
};

const styles = StyleSheet.create({
  container: { backgroundColor: '#f4f4f6', flex: 1 },
  text: { fontSize: 16, fontWeight: '600', color: '#333' },
  card: { flex: 1, height: 100, backgroundColor: '#007AFF', borderRadius: 8
},
});

```

## 2. The `Dimensions` API (Legacy / Outside Component Flow)

The `Dimensions` API is a static object used to fetch initial screen dimensions.

- **Why:** It retrieves screen dimensions immediately at module load time.
- **When to use:** Use it **outside component renders** (e.g., initial constants or style sheets) or in utility/helper functions.
- **The Drawback:** Unlike `useWindowDimensions`, `Dimensions.get()` **does NOT trigger a re-render** when the screen rotates unless you manually attach a `change` event listener.

TypeScript

```

import { Dimensions } from 'react-native';

// Get window dimensions (excludes system bars)
const { width: SCREEN_WIDTH, height: SCREEN_HEIGHT } =
Dimensions.get('window');

// Useful for static global scale factors or constants
export const IS_TABLET = SCREEN_WIDTH >= 768;
export const CARD_WIDTH = (SCREEN_WIDTH - 32) / 2; // 2-column layout width
minus padding

```

## 3. Comparison: `Dimensions` VS. `useWindowDimensions`

| Feature | <code>Dimensions.get()</code> | <code>useWindowDimensions()</code> |
|---------|-------------------------------|------------------------------------|
| Type    | Static Imperative API         | React Hook                         |

| Feature                            | Dimensions.get()                                                | useWindowDimensions()                                        |
|------------------------------------|-----------------------------------------------------------------|--------------------------------------------------------------|
| <b>Listens to Screen Rotation?</b> | ✗ No (Requires manual event listener)                           | ✓ Yes (Auto-triggers re-render)                              |
| <b>Usable outside components?</b>  | ✓ Yes                                                           | ✗ No (Hook rule applies)                                     |
| <b>Best For</b>                    | Static constants, styling calculations outside React components | Responsive component UI, orientation logic, responsive grids |

## 4. Best Practices for Responsive Layouts

### A. Use Percentage Dimensions & Flexbox First

Before calculating exact pixel numbers using screen width, check if pure CSS percentages or Flexbox can solve the layout cleanly.

TypeScript

```
// Percentage width adjusts automatically without hooks
<View style={{ width: '90%', alignSelf: 'center' }} />
```

### B. Scaled Sizes for Typography and Spacing

For consistent design systems across small devices (e.g., iPhone SE) and large phones (e.g., iPhone Pro Max), create a scaling helper based on a standard design width (typically 375px or 390px):

TypeScript

```
import { Dimensions, PixelRatio } from 'react-native';

const { width: SCREEN_WIDTH } = Dimensions.get('window');

// Standard design baseline (e.g., Figma artboard width of 375px)
const BASE_WIDTH = 375;

// Scale helper for font sizes, margins, and padding
export const scale = (size: number) => {
```

```

    const scaleFactor = SCREEN_WIDTH / BASE_WIDTH;
    const newSize = size * scaleFactor;
    return Math.round(PixelRatio.roundToNearestPixel(newSize));
  };

  // Usage inside styles:
  // fontSize: scale(16),
  // marginVertical: scale(12)

```

To handle edge-to-edge screen elements like camera notches, dynamic islands, curved corners, and home indicator bars cleanly across iOS and Android, the community standard is **react-native-safe-area-context**.

Here is how to set it up and use it effectively in your project.

## 1. Setup & Installation

Install the package using your package manager:

**Bash**

```

# npm
npm install react-native-safe-area-context

# yarn
yarn add react-native-safe-area-context

# Expo (recommended for Expo apps)
npx expo install react-native-safe-area-context

```

**Note:** If you are using React Native CLI (non-Expo), remember to run `pod install` inside the `ios/` folder afterward.

## 2. Wrap Your App with `SafeAreaProvider`

You must wrap the root level of your component tree (typically in `App.tsx` or `_layout.tsx`) with **`SafeAreaProvider`**. This measures the device's safe bounds natively and distributes the values down your component tree.

**TypeScript**

```

import React from 'react';
import { SafeAreaProvider } from 'react-native-safe-area-context';
import HomeScreen from './HomeScreen';

export default function App() {
  return (
    <SafeAreaProvider>
      <HomeScreen />
    </SafeAreaProvider>
  );
}

```

```
}
```

### 3. Pattern 1: Component Approach (`SafeAreaView`)

The simplest way to protect content from overlapping notches or home bars is by replacing standard `<View>` containers with `<SafeAreaView>`.

- **When to use:** Full-screen views or simple top-level screen components.
- **Property control:** Use the `edges` prop to choose which boundaries need padding.

#### TypeScript

```
import React from 'react';
import { Text, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';

export default function SimpleScreen() {
  return (
    // Applies padding ONLY to top (notch) and bottom (home bar)
    <SafeAreaView style={styles.container} edges={['top', 'bottom']}>
      <Text style={styles.text}>Safe from notches and home bars!</Text>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#1a1a1a',
  },
  text: {
    color: 'white',
    fontSize: 18,
  },
});
```

### 4. Pattern 2: Hook Approach (`useSafeAreaInsets`) — Most Flexible

For full control—such as setting background colors that extend into the notch area while padding content underneath—use the `useSafeAreaInsets` hook.

The hook returns an object with exact pixel measurements: `{ top, bottom, left, right }`.

#### TypeScript

```
import React from 'react';
import { View, Text, StyleSheet, TouchableOpacity } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

export default function FlexibleScreen() {
  const insets = useSafeAreaInsets();
```

```

return (
  <View style={styles.container}>
    {/*
      Custom Header:
      Background color extends into the status bar/notch area,
      while padding pushes content down safely below the notch.
    */}
    <View style={[styles.header, { paddingTop: insets.top }]}>
      <Text style={styles.headerTitle}>Header Title</Text>
    </View>

    <View style={styles.content}>
      <Text>Screen Body Content</Text>
    </View>

    {/*
      Floating Action Button:
      Positioned safely above the Home Indicator bar at the bottom.
    */}
    <TouchableOpacity
      style={[styles.floatingButton, { bottom: insets.bottom + 16 }]}
    >
      <Text style={styles.buttonText}>Action</Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: 'rgba(0,0,0,0.1)',
  },
  header: {
    backgroundColor: 'black',
    paddingBottom: 16,
    paddingHorizontal: 20,
  },
  headerTitle: {
    color: 'white',
    fontSize: 20,
    fontWeight: 'bold',
  },
  content: {
    flex: 1,
    padding: 20,
  },
  floatingButton: {
    position: 'absolute',
    right: 20,
    backgroundColor: 'black',
    paddingVertical: 14,
    paddingHorizontal: 24,
    borderRadius: 30,
  },
  buttonText: {

```



```

        color: '#ffffff',
        fontWeight: 'bold',
    },
});

```

## Summary Comparison & Rules of Thumb

| Approach                   | Best Used For                                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>SafeAreaView</b>        | Full-screen views where simple container padding is all you need.                                                              |
| <b>useSafeAreaInsets()</b> | Custom navigation bars, absolute positioning (badges, FABs), full-bleed background colors, or fine-tuned padding calculations. |

**Pro Tip:** Avoid nesting multiple `SafeAreaView` components inside each other. If a parent view applies safe area padding, child views do not need to apply it again.